xonsh Excursions

Rohit Goswami · 2024-07-22 · ramblings · tools · shell

Background

Recently I found myself writing a bunch of search and replace one-liners.

```
export FROM='Matrix3d'; export TO='Matrix3S'; ag -l $FROM | xargs -I {} sd $FROM $TO {}
```

Which works, especially since both `ag` and `sd` are rather good, but it is still:

- Slightly non-ergonomic to type
- Difficult to keep track of
  - Modulo dumping everything in a `.sh` file

These reminded me of the rich set of alternate shells[^fn:1].

Reaching for `xonsh`

Although `nushell`, `elvish` and even `oil` seemed promising, I settled on the Python based xonsh.

```
pipx install xonsh[full]
pipx inject xonsh xcontrib-sh
echo 'xontrib load sh' >> ~/.xonshrc
```

Where we use the injection mechanism to add the xcontrib-sh plugin for running shell commands without using `xonsh`, since we might not want to translate every command to be compliant with `xonsh`.

Additions

- `xonsh-mode` works well with `emacs` and is on MELPA
- xxh makes it pretty trivial to take into foreign SSH machines

Scripting substitutions

Directly using `xonsh` we can rewrite the earlier `bash` script into:

```
from pathlib import Path

def replace_make_shared():
    FROM_MAKE = 'Matter'
    TO_MAKE = 'Matter'

    files = $(ag -l @(FROM_MAKE)).splitlines()

    for f in files:
        if Path(f).exists():
            sd @(FROM_MAKE) @(TO_MAKE) @(f)
        else:
            echo "File not found: @(file)"

replace_make_shared()
```

Using Python

It is instructive to recall how this would work in pure `python`.

```
import subprocess
from pathlib import Path

def replace_make_shared():
    FROM_MAKE = 'std::shared_ptr<Matter>'
    TO_MAKE = 'Matter*'

    result = subprocess.run(['ag', '-l', FROM_MAKE],
                            capture_output=True, text=True)
    files = result.stdout.splitlines()
    actionable_files = [x for x in files if 'migrator' not in x]

    for f in actionable_files:
        file_path = Path(f)
        if file_path.exists():
            subprocess.run(['sd', FROM_MAKE, TO_MAKE,
                            str(file_path)], check=True)
        else:
            print(f"File not found: {file_path}")

replace_make_shared()
```

With sh, shell calls become more ergonomic, however, it is still easier to interoperate between the shell outputs and Python code (e.g. using `echo`) using `xonsh`.

Conclusions

Personally, `xonsh` hits the sweet-spot of being slightly less finicky than POSIX shells while being less verbose than the pure `python` variant. It isn't perfect though:

- The `python` dependence, even with `pipx` can be an issue [^fn:2]
  - Packages which are needed have to be injected into the same environment..
    - `nix` maybe...
- There is no good testing framework
  - Ugly `subprocess` based `pytest` tests could be written

For now, though, this is sufficiently advantageous.